

Introducing Isabelle/HOL:-

→ KEATS

→ Higher-order logic

* logic $\forall, \exists, \vee, \wedge, \rightarrow, \dots$

* functional programming

→ features

* define datatypes

* functions $\begin{cases} \rightarrow \text{recursive} \\ \rightarrow \text{non-} \end{cases} \rightarrow \text{functional programming}$

* logical properties of functions $\begin{cases} \rightarrow \text{state} \\ \rightarrow \text{prove} \end{cases}$

→ functional programming concepts

* Currying

E.g. maths textbook $\Rightarrow \underline{g(x, y)}, \underline{h(x, y, z)}$

Curried version:-

$g \ x \ y$

$h \ x \ y \ z$

→ ~~*~~ lambda expressions

E.g. maths textbook $\Rightarrow f(x, y) = 2x + y$

$f' = (\lambda x \ y. 2x + y)$

$f(a, b) = 2a + b$

$f' a \ b$ $= 2a + b$

$f' a = (\lambda y. 2a + y)$

* polymorphic types

• f takes a real number and returns real

$f: \text{real} \rightarrow \text{real}$

• $f: (\text{real list}) \rightarrow \text{real}$

• $f: ('a \text{ list}) \rightarrow 'a$

$\swarrow \quad \searrow$
real nat int

→ Terms

numbers, functions, lists, trees --

True, false

0, 1, ... $\Rightarrow$ overloaded $\begin{matrix} \nearrow \text{nat} \\ \rightarrow \text{int} \\ \searrow \text{rational} \end{matrix}$
(+), (-), (*) $\nearrow$

→ type annotate terms

$\tau :: \text{bool} \mid \text{nat} \mid \text{int} \mid \dots$ base types

$\mid 'a \mid 'b \mid 'c \mid \dots$ polymorphic

$\mid \tau \Rightarrow \tau$ function

e.g. $f :: \text{nat} \Rightarrow \text{nat}$

$f' :: \text{nat} \Rightarrow (\text{nat} \Rightarrow \text{nat})$

$\mid \tau \times \tau$ product

E.g. $f :: (\text{nat} \times \text{nat}) \Rightarrow \text{nat}$

E.g. $\text{fst} :: ('a \times 'b) \Rightarrow 'a$

$\text{snd} :: ('a \times 'b) \Rightarrow 'b$

$\mid \tau \text{ list} \rightarrow \text{append}(\tau), \text{hd}, \text{tail}$

$\mid \tau \text{ set}$

{ ... } user. defined

• Note :: blue terms are undefined

• notational numbers in Isabelle are either "0", or "Suc a", where a is another natural number

• lists are either empty -- []

non-empty $\Leftrightarrow \text{Cons } x \ xs$

$[a, b, c] \rightsquigarrow \text{Cons } a \ (\text{Cons } b \ (\text{Cons } c \ []))$

→ specify logical properties of terms

* of type bool

E.g. true :: bool

• true ∧ false :: bool

• [a, b] ≠ [c]

• a = b → [a] = [b]

* theorem statements

• $A, B \vdash C$

• $X, Y \vdash Z$

• $X \Rightarrow Y, X \vdash Y \Rightarrow$ modus-ponens

In Isabelle: meta-logic

lemma "A $\Rightarrow$ B $\Rightarrow$ C"

lemma "[A; B] $\Rightarrow$ C"

Note: "sorry" can be used to "prove" a lemma that you don't know how to prove

Top-down proofs

- Next: - how to prove theorems in Isabelle
HoC.